SHA-256 File/Data Integrity Checker

Cyber and Network Security (CNS) Python Project Report

Student: Aayushi Gurav
Programming Language: Python
Interface: Tkinter

1. Introduction

Data integrity is an important security property that ensures information has not been changed without authorization. Cryptographic hash functions provide a practical mechanism for detecting such changes. This project implements SHA-256 based file integrity verification in Python.

2. Problem Statement

Files may be modified accidentally or maliciously during storage, transfer, or handling. Users need a simple method to determine whether a file is identical to a trusted version. The proposed application calculates a SHA-256 digest and compares it with a trusted reference value.

3. Objectives

The objectives are to implement SHA-256 hashing, calculate file digests, compare trusted and current hashes, detect content modification, validate user input, and provide a user-friendly GUI.

4. Technologies Used

Python 3, Tkinter, hashlib, pathlib, re. The implementation uses only Python standard-library modules, so no third-party installation is required.

5. Cryptography / CNS Concepts

The project demonstrates cryptographic hashing, SHA-256, fixed-length message digests, integrity verification, collision-resistance concept, input validation, and secure file handling. SHA-256 is one-way and is not encryption.

6. System Design

Flow: Select File → Validate Path → Read File in Chunks → SHA-256 Update → Generate Digest → Save Trusted Reference → Recalculate Later → Compare Hashes → Display Integrity Result.

7. Implementation

The application uses hashlib.sha256(). The file is opened in binary read-only mode and processed in 1 MB chunks. The digest is represented as 64 hexadecimal characters. A 500 MB configurable file-size limit is used as a basic resource-safety measure.

8. Key Features

File selection, SHA-256 calculation, trusted hash export, reference hash loading, hash comparison, clear verification status, invalid-hash detection, file-size validation, and read-only file processing.

9. Testing and Results

The application should be tested with an unchanged file, a modified file, an empty file, an invalid hash, no selected file, and an oversized file. Matching hashes produce an Integrity Verified result; mismatching hashes produce an Integrity Check Failed result.

10. Security Considerations

The application does not execute selected files or alter their contents. Hashes are validated before comparison. Chunked processing reduces memory usage. A trusted reference hash must itself be protected; SHA-256 alone

does not authenticate the source of a file.

11. Future Scope

Possible extensions include folder scanning, batch verification, digital signatures, HMAC authentication, audit logs, drag-and-drop support, and a web interface.

12. Conclusion

The SHA-256 File/Data Integrity Checker is a practical CNS prototype that demonstrates how cryptographic hashing can detect changes in digital data. It connects a fundamental cryptography concept with a real-world file-security use case.

Sample SHA-256 Verification Logic

```
trusted_hash = calculate_sha256(original_file)
current_hash = calculate_sha256(file)

if trusted_hash == current_hash:
    print("Integrity Verified")
else:
    print("Modification Detected")
```

Note: Add actual application screenshots and your final GitHub URL before submission.